Phase 6: Express 백엔드

웹 풀스택 개발 과정

Day 24 ~ Day 32

Day 24 - Node.js 첫 서버

> **Phase 6: Express** | 학습일: 24일차

왜 Node.js를 배우는가?

지금까지 우리는 **브라우저** 안에서 JavaScript를 실행했습니다. HTML 페이지를 열고, 버튼을 클릭하면 JavaScript가 동작했죠.

하지만 실제 웹 서비스에는 **서버**가 필요합니다.

> **실생활 비유**: 음식 배달앱을 생각해보세요.
>
> - **프론트엔드(브라우저)** = 배달앱 화면 (메뉴 보기, 주문 버튼)
> - **백엔드(서버)** = 실제 주방 (주문 접수, 요리, 배달 관리)
>
> 아무리 예쁜 앱을 만들어도, 주방(서버)이 없으면 음식(데이터)을 줄 수 없습니다!

Node.js란?

Node.js는 Chrome의 V8 엔진을 기반으로 한 **JavaScript 런타임**입니다. 쉽게 말하면, 브라우저 없이도 JavaScript를 실행할 수 있게 해주는 프로그램입니다.

```
브라우저 JavaScript → DOM 조작, 사용자 인터페이스, 클릭 이벤트
Node.js JavaScript  → 파일 읽기/쓰기, 서버 만들기, 데이터베이스 연결
```

http 모듈로 서버 만들기

Node.js에는 웹 서버를 만들 수 있는 `http` 모듈이 내장되어 있습니다.

```javascript
// server.js - 나의 첫 번째 서버!
const http = require('http');  // Node.js 내장 http 모듈 불러오기

// 서버 생성
// createServer에 전달하는 함수는 "요청이 올 때마다" 실행됩니다
const server = http.createServer((req, res) => {
  // req = request (요청) - 클라이언트가 보낸 정보
  // res = response (응답) - 서버가 보내는 정보

  // 응답 헤더 설정 - "나는 JSON 데이터를 보낼 거야"
  res.setHeader('Content-Type', 'application/json');

  // 응답 본문 전송
  res.end(JSON.stringify({
    message: '할일 API 서버입니다',
    version: '1.0.0'
  }));
});

// 서버 시작 - 3000번 포트에서 대기
server.listen(3000, () => {
  console.log('서버가 http://localhost:3000 에서 실행 중입니다');
});
```

> **실생활 비유**: `createServer` 는 **편의점 카운터**와 같습니다.
>
> - 손님(req)이 들어오면 → 카운터 직원이 응대(콜백 함수 실행)
> - 직원이 물건을 건네줌(res.end) → 손님이 받아감
> - `listen(3000)` 은 "3000번 문을 열고 손님을 기다려라"는 의미

요청(req) 객체의 주요 속성

속성	설명	예시
`req.url`	요청한 URL 경로	`/api/todos`
`req.method`	HTTP 메서드	`GET`, `POST`, `PUT`, `DELETE`

URL과 메서드에 따라 다른 응답 보내기

```
const server = http.createServer((req, res) => {
  res.setHeader('Content-Type', 'application/json');

  // URL과 메서드에 따라 다르게 처리 (이것이 "라우팅")
  if (req.url === '/api/todos' && req.method === 'GET') {
    // 할일 목록 조회
    res.end(JSON.stringify(todos));

  } else if (req.url === '/api/todos' && req.method === 'POST') {
    // 할일 추가 (아래에서 설명)
    // ...

  } else {
    // 알 수 없는 경로 → 404 에러
    res.statusCode = 404;
    res.end(JSON.stringify({ error: '찾을 수 없는 경로입니다' }));
  }
});
```

POST 요청의 body 읽기

GET 요청은 URL만 있으면 되지만, POST 요청은 **데이터(body)**를 함께 보냅니다. Node.js에서는 body를 조각(chunk)으로 받아야 합니다.

```
if (req.method === 'POST') {
  let body = '';  // 빈 문자열로 시작

  // 데이터가 조각으로 들어올 때마다 이어붙이기
  req.on('data', (chunk) => {
    body += chunk;  // "안" + "녕하" + "세요" → "안녕하세요"
  });

  // 모든 데이터를 다 받았을 때
  req.on('end', () => {
    const { title } = JSON.parse(body);  // 문자열 → 객체로 변환
    const newTodo = {
      id: todos.length + 1,
      title: title,
      done: false
    };
    todos.push(newTodo);

    res.statusCode = 201;  // 201 = 새로 생성됨
    res.end(JSON.stringify(newTodo));
  });
}
```

> **자주 하는 실수**: `req.on('data')` 를 빼먹고 바로 `JSON.parse(body)` 를 하면 빈 문자열을 파싱하게 되어 에러가 납니다. body는 반드시 `data` + `end` 이벤트로 읽어야 합니다.

fs 모듈 - 파일 읽기/쓰기

Node.js에서는 파일 시스템에 접근할 수 있습니다.

```node
const fs = require('fs');

// 파일 읽기 (동기 방식 - 파일을 다 읽을 때까지 기다림)
const data = fs.readFileSync('todos.json', 'utf-8');
console.log(data);  // 파일 내용 출력

// 파일 쓰기 (동기 방식)
const todos = [{ id: 1, title: '공부하기', done: false }];
fs.writeFileSync('todos.json', JSON.stringify(todos, null, 2));
// JSON.stringify의 세 번째 인자 2는 들여쓰기 칸 수

// 파일 존재 확인
if (fs.existsSync('todos.json')) {
  console.log('파일이 있습니다');
} else {
  console.log('파일이 없습니다');
}
```

실행 방법

```terminal
# 서버 실행
node server.js

# 다른 터미널에서 테스트
curl http://localhost:3000/api/todos
curl -X POST http://localhost:3000/api/todos \
  -H "Content-Type: application/json" \
  -d '{"title": "Node.js 공부하기"}'
```

정리

개념	핵심
Node.js	브라우저 밖에서 JS 실행하는 런타임
http 모듈	`createServer()` + `listen()` 으로 서버 생성
req	요청 객체 (`url` , `method` , `on('data')`)
res	응답 객체 (`setHeader` , `statusCode` , `end`)
fs 모듈	파일 읽기(`readFileSync`), 쓰기(`writeFileSync`)
라우팅	URL + 메서드 조합으로 분기 처리

Day 25 - 모듈 시스템과 파일 저장

> **Phase 6: Express** | 학습일: 25일차

왜 모듈이 필요한가?

프로그램이 커지면 모든 코드를 한 파일에 넣기 어렵습니다.

> **실생활 비유**: 회사 조직도를 생각해보세요.
>
> - 1인 기업 → 모든 일을 혼자 (파일 1개)
> - 10명 회사 → 마케팅팀, 개발팀, 영업팀 (파일 여러 개로 분리)
> - 각 팀이 자기 역할만 하고, 필요할 때 다른 팀에 요청

CommonJS vs ESM

Node.js에는 두 가지 모듈 시스템이 있습니다.

CommonJS (Node.js 기본 방식)

```
// math.js - 모듈 만들기 (내보내기)
function add(a, b) {
  return a + b;
}

function multiply(a, b) {
  return a * b;
}

// 다른 파일에서 쓸 수 있도록 내보내기
module.exports = { add, multiply };
```

```
// app.js - 모듈 가져오기
const { add, multiply } = require('./math');  // .js 생략 가능

console.log(add(2, 3));        // 5
console.log(multiply(4, 5));   // 20
```

ESM (최신 표준 방식)

```
// math.mjs - ESM 방식으로 내보내기
export function add(a, b) {
  return a + b;
}

export function multiply(a, b) {
  return a * b;
}
```

```
// app.mjs - ESM 방식으로 가져오기
import { add, multiply } from './math.mjs';

console.log(add(2, 3));  // 5
```

구분	CommonJS	ESM
문법	`require()` / `module.exports`	`import` / `export`
확장자	`.js`	`.mjs` 또는 package.json에 `"type": "module"`
현재 상태	Node.js 기본	최신 표준 (브라우저와 동일)

> **초보자 팁**: 현재 대부분의 Node.js 프로젝트는 CommonJS를 사용합니다. React/Next.js에서는 ESM을 쓰지만, Express 서버에서는 CommonJS가 일반적입니다.

path 모듈 - 안전한 경로 처리

운영체제마다 파일 경로 구분자가 다릅니다 (Windows: `\`, Mac/Linux: `/`). `path` 모듈은 이를 자동으로 처리합니다.

```
const path = require('path');

// __dirname: 현재 파일이 있는 폴더의 절대 경로
console.log(__dirname);  // /Users/student/project

// path.join: 경로를 안전하게 연결
const filePath = path.join(__dirname, 'data', 'todos.json');
// Mac: /Users/student/project/data/todos.json
// Win: C:\Users\student\project\data\todos.json

// path.extname: 확장자 추출
console.log(path.extname('report.pdf'));  // '.pdf'
```

파일 기반 데이터 영속화

Day 24의 서버는 메모리에만 데이터를 저장했기 때문에, 서버를 재시작하면 모든 데이터가 사라졌습니다. JSON 파일에 저장하면 이 문제를 해결할 수 있습니다.

```
const fs = require('fs');
const path = require('path');

const FILE_PATH = path.join(__dirname, 'todos.json');

// 파일에서 할일 불러오기
function loadTodos() {
  try {
    // 파일 읽기 시도
    const data = fs.readFileSync(FILE_PATH, 'utf-8');
    return JSON.parse(data);  // 문자열 → 배열
  } catch (error) {
    // 파일이 없으면 빈 배열 반환 (처음 실행할 때)
    return [];
  }
}

// 파일에 할일 저장하기
function saveTodos(todos) {
  // JSON.stringify(데이터, null, 2) → 보기 좋게 들여쓰기
  fs.writeFileSync(FILE_PATH, JSON.stringify(todos, null, 2));
}

// 사용 예시
let todos = loadTodos();            // 파일에서 불러오기
todos.push({ id: 1, title: '공부' });
saveTodos(todos);                   // 파일에 저장하기
// → 서버를 재시작해도 데이터가 유지됩니다!
```

npm 패키지 활용

외부 라이브러리를 사용하면 개발이 훨씬 편해집니다.

```
// lodash - 유틸리티 함수 모음
const _ = require('lodash');
_.chunk([1, 2, 3, 4, 5], 2);       // [[1, 2], [3, 4], [5]]
_.groupBy(todos, 'category');       // 카테고리별로 그룹화

// dayjs - 날짜 처리 (moment.js보다 가벼움)
const dayjs = require('dayjs');
dayjs().format('YYYY-MM-DD HH:mm:ss');  // "2026-03-05 14:30:00"

// uuid - 고유 ID 생성
const { v4: uuidv4 } = require('uuid');
const id = uuidv4();  // "550e8400-e29b-41d4-a716-446655440000"
// 숫자 ID와 달리 충돌 가능성이 거의 없음
```

정리

개념	핵심
CommonJS	`require()` / `module.exports` — Node.js 기본
ESM	`import` / `export` — 최신 표준
path 모듈	`path.join(__dirname, ...)` 안전한 경로
영속화	`loadTodos()` / `saveTodos()` 패턴
lodash	유틸리티 함수 (groupBy, chunk 등)
uuid	충돌 없는 고유 ID 생성

Day 26 - npm 패키지 활용

> **Phase 6: Express** | 학습일: 26일차

npm이란?

npm(Node Package Manager)은 전 세계 개발자들이 만든 패키지(라이브러리)를 설치하고 관리하는 도구입니다.

> **실생활 비유**: npm은 **앱스토어**와 같습니다.
>
> - 앱스토어에서 앱을 검색하고 설치하듯이
> - npm에서 패키지를 검색하고 설치합니다
> - 현재 200만 개 이상의 패키지가 등록되어 있습니다!

package.json — 프로젝트 설명서

```
# 프로젝트 초기화 (package.json 자동 생성)
npm init -y
```

```json
{
  "name": "todo-server",          // 프로젝트 이름
  "version": "1.0.0",             // 버전
  "description": "할일 API 서버", // 설명
  "main": "server.js",            // 진입점 파일
  "scripts": {                    // 실행 명령어 (단축키)
    "start": "node server.js",    // npm start → node server.js
    "dev": "nodemon server.js"    // npm run dev → nodemon server.js
  },
  "dependencies": {               // 실제 서비스에 필요한 패키지
    "dotenv": "^16.4.0",
    "uuid": "^9.0.0"
  },
  "devDependencies": {            // 개발할 때만 필요한 패키지
    "nodemon": "^3.1.0"
  }
}
```

> **dependencies vs devDependencies**
>
> - `dependencies` : 서비스 운영에 필요 (배달앱의 결제 시스템)
> - `devDependencies` : 개발할 때만 필요 (배달앱의 테스트 도구)

dotenv — 환경 변수 관리

비밀번호, API 키 같은 민감한 정보를 코드에 직접 쓰면 위험합니다.

> **실생활 비유**: 금고 비밀번호를 벽에 써놓는 사람은 없죠? `.env` 파일은 **별도의 금고**에 비밀번호를 보관하는 것과 같습니다.

```
# .env 파일 (이 파일은 절대 GitHub에 올리지 않습니다!)
PORT=3000
DATA_DIR=./data
DB_PASSWORD=super_secret_123
```

```
// server.js
require('dotenv').config();  // .env 파일의 내용을 process.env에 로드

const PORT = process.env.PORT || 3000;         // 환경변수에서 읽기
const DATA_DIR = process.env.DATA_DIR || './data';

console.log(`서버 포트: ${PORT}`);              // "서버 포트: 3000"
console.log(`데이터 폴더: ${DATA_DIR}`);        // "데이터 폴더: ./data"
```

자주 하는 실수:

- `.env` 파일을 Git에 올리지 마세요! (`.gitignore` 에 추가)
- `.env.example` 파일에 키 이름만 적어서 공유하세요
- `require('dotenv').config()` 는 파일 맨 위에 써야 합니다

nodemon — 자동 재시작

코드를 수정할 때마다 서버를 껐다 켜는 건 귀찮습니다.

```
# nodemon 설치 (개발용)
npm install --save-dev nodemon

# 실행
npx nodemon server.js
# 또는 scripts에 등록 후
npm run dev
```

`nodemon` 은 파일 변경을 감지해서 자동으로 서버를 재시작합니다.

정리

개념	핵심
npm init	`package.json` 생성
npm install	패키지 설치 (dependencies에 기록)
scripts	`npm start`, `npm run dev` 명령어 단축키
dotenv	`.env` → `process.env` 로 환경 변수 관리
nodemon	파일 변경 감지 → 자동 재시작

Day 27 - Express 서버

> **Phase 6: Express** | 학습일: 27일차

왜 Express를 쓰는가?

Day 24에서 `http` 모듈로 서버를 만들어봤는데, URL마다 `if/else`로 분기하는 게 복잡했죠?

> **실생활 비유**:
>
> - `http` 모듈 = **자전거** (직접 페달 밟기, 기어 변속, 브레이크)
> - Express = **자동차** (핸들만 돌리면 알아서 움직임)
>
> 둘 다 목적지에 도착하지만, Express가 훨씬 편합니다!

Before (http 모듈) vs After (Express)

```
// Before: http 모듈 - 복잡한 if/else
if (req.url === '/api/todos' && req.method === 'GET') {
  res.setHeader('Content-Type', 'application/json');
  res.end(JSON.stringify(todos));
} else if (req.url === '/api/todos' && req.method === 'POST') {
  let body = '';
  req.on('data', chunk => body += chunk);
  req.on('end', () => { /* ... 복잡 ... */ });
}
```

```
// After: Express - 깔끔하고 직관적
app.get('/api/todos', (req, res) => {
  res.json(todos);  // 자동으로 JSON 변환 + 헤더 설정
});

app.post('/api/todos', (req, res) => {
  const { title } = req.body;  // body가 자동으로 파싱됨!
  // ...
});
```

Express 기본 구조

```
const express = require('express');
const app = express();  // Express 앱 생성

// 미들웨어 등록
app.use(express.json());  // JSON body를 자동으로 파싱

// 라우트 정의
app.get('/', (req, res) => {
  res.json({ message: '할일 API 서버입니다' });
});

// 서버 시작
app.listen(3000, () => {
  console.log('서버가 3000번 포트에서 실행 중');
});
```

미들웨어 (Middleware)

미들웨어는 요청(req)과 응답(res) **사이에서** 실행되는 함수입니다.

> **실생활 비유**: **공항 보안 검색대**를 생각하세요.
>
> 1. 신분증 확인 (인증 미들웨어)
> 2. 짐 검사 (데이터 검증 미들웨어)
> 3. 금속 탐지기 (보안 미들웨어)
> 4. 탑승 (라우트 핸들러)
>
> 각 단계를 통과해야 다음 단계로 갈 수 있습니다!

```
요청 → [cors] → [express.json()] → [로깅] → [라우트 핸들러] → 응답
```

```
// 커스텀 로깅 미들웨어
app.use((req, res, next) => {
  console.log(`[${new Date().toLocaleString()}] ${req.method} ${req.url}`);
  next();  // ← 이것을 호출해야 다음 미들웨어로 넘어갑니다!
});

// next()를 안 부르면? → 요청이 여기서 멈추고 응답이 안 갑니다!
```

자주 쓰는 미들웨어

```
const cors = require('cors');

app.use(cors());            // 다른 도메인에서의 요청 허용
app.use(express.json());    // JSON body 자동 파싱
```

Router로 라우트 분리

파일 하나에 모든 라우트를 넣으면 코드가 너무 길어집니다.

```
// routes/todos.js — 할일 관련 라우트만 모아놓기
const express = require('express');
const router = express.Router();

// 이 파일 안에서는 '/api/todos' 이후의 경로만 씀
router.get('/', (req, res) => {
  // GET /api/todos
  res.json(todos);
});

router.post('/', (req, res) => {
  // POST /api/todos
  const { title } = req.body;
  // ...
  res.status(201).json(newTodo);
});

router.get('/:id', (req, res) => {
  // GET /api/todos/5 → req.params.id === "5"
  const todo = todos.find(t => t.id === Number(req.params.id));
  if (!todo) return res.status(404).json({ error: '할일을 찾을 수 없습니다' });
  res.json(todo);
});

router.put('/:id', (req, res) => {
  // PUT /api/todos/5
  const { id } = req.params;
  const { title, done } = req.body;
  // ...
});

router.delete('/:id', (req, res) => {
  // DELETE /api/todos/5
  // ...
  res.status(204).end();  // 204 = 성공했지만 보낼 내용 없음
});

module.exports = router;
```

```
// index.js — 메인 서버 파일
const todoRouter = require('./routes/todos');

// '/api/todos'로 시작하는 요청은 todoRouter가 처리
app.use('/api/todos', todoRouter);
```

라우팅 패턴

```
// 1. 경로 파라미터 (URL의 일부)
app.get('/api/todos/:id', (req, res) => {
  const { id } = req.params;  // URL에서 추출
  // /api/todos/5 → id = "5"
});

// 2. 쿼리 스트링 (? 뒤의 값)
app.get('/api/todos', (req, res) => {
  const { done, page, limit } = req.query;  // URL 파라미터
  // /api/todos?done=true&page=1 → done = "true", page = "1"
});

// 3. 라우트 체이닝 (같은 경로, 다른 메서드)
router.route('/:id')
  .get(getById)       // GET /api/todos/:id
  .put(update)         // PUT /api/todos/:id
  .delete(remove);     // DELETE /api/todos/:id
```

REST API 설계

메서드	경로	설명	상태코드
GET	/api/todos	목록 조회	200
GET	/api/todos/:id	상세 조회	200 / 404
POST	/api/todos	추가	201
PUT	/api/todos/:id	수정	200 / 404
DELETE	/api/todos/:id	삭제	204 / 404

MVC 패턴

큰 프로젝트에서는 코드를 역할별로 분리합니다.

> **실생활 비유**: 식당을 생각해보세요.
>
> - **Model (모델)** = 주방: 재료(데이터)를 관리하고 요리(처리)
> - **View (뷰)** = 메뉴판/서빙: 손님에게 보여주는 부분
> - **Controller (컨트롤러)** = 웨이터: 주문 받고 주방에 전달

```
요청 → Controller (주문 접수) → Model (데이터 처리) → Controller → 응답
```

정리

개념	핵심
Express	Node.js 웹 프레임워크 (간결한 라우팅)
미들웨어	요청→응답 사이 함수 (`next()` 필수)
Router	라우트를 파일로 분리 (`express.Router()`)
req.params	URL 파라미터 (`/todos/:id`)
req.query	쿼리 스트링 (`?done=true`)
req.body	POST/PUT의 요청 데이터
MVC	Model-View-Controller 역할 분리

Day 28 - 데이터베이스 연동

> **Phase 6: Express** | 학습일: 28일차

왜 데이터베이스가 필요한가?

Day 25에서 JSON 파일에 데이터를 저장했는데, 이 방식에는 한계가 있습니다:

- 동시에 여러 사용자가 접근하면 데이터가 꼬일 수 있음
- 데이터가 많아지면 파일 읽기/쓰기가 느려짐
- 검색, 정렬, 필터링이 불편함

> **실생활 비유**:
>
> - JSON 파일 = **종이 노트**에 할일 적기 (간단하지만 찾기 어려움)
> - 데이터베이스 = **엑셀 스프레드시트** (정렬, 필터, 검색 가능)

better-sqlite3로 Express + SQLite 연동

```
// database.js — DB 연결 및 초기화
const Database = require('better-sqlite3');
const path = require('path');

// DB 파일 생성 (없으면 자동 생성됨)
const db = new Database(path.join(__dirname, 'todos.db'));

// 테이블 생성 (IF NOT EXISTS: 이미 있으면 무시)
db.exec(`
  CREATE TABLE IF NOT EXISTS todos (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    done INTEGER DEFAULT 0,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
  )
`);

module.exports = db;
```

```
// routes/todos.js — DB를 사용하는 라우트
const db = require('../database');

// 전체 조회
router.get('/', (req, res) => {
  const todos = db.prepare('SELECT * FROM todos ORDER BY created_at DESC').all();
  res.json(todos);
});

// 단일 조회
router.get('/:id', (req, res) => {
  const todo = db.prepare('SELECT * FROM todos WHERE id = ?').get(req.params.id);
  //                                                          ↑ SQL 인젝션 방지!
  if (!todo) return res.status(404).json({ error: '할일을 찾을 수 없습니다' });
  res.json(todo);
});

// 추가
router.post('/', (req, res) => {
  const { title } = req.body;
  if (!title) return res.status(400).json({ error: 'title은 필수입니다' });

  const result = db.prepare('INSERT INTO todos (title) VALUES (?)').run(title);
  const newTodo = db.prepare('SELECT * FROM todos WHERE id = ?').get(result.lastI
  res.status(201).json(newTodo);
});

// 수정
router.put('/:id', (req, res) => {
  const { title, done } = req.body;
  const result = db.prepare(
    'UPDATE todos SET title = COALESCE(?, title), done = COALESCE(?, done) WHERE
  ).run(title, done, req.params.id);
  // COALESCE: 값이 null이면 기존값 유지 (부분 수정 가능!)

  if (result.changes === 0) return res.status(404).json({ error: '할일을 찾을 수 없습
  const todo = db.prepare('SELECT * FROM todos WHERE id = ?').get(req.params.id);
  res.json(todo);
});

// 삭제
router.delete('/:id', (req, res) => {
  const result = db.prepare('DELETE FROM todos WHERE id = ?').run(req.params.id);
  if (result.changes === 0) return res.status(404).json({ error: '할일을 찾을 수 없습
  res.status(204).end();
});
```

Prepared Statement가 중요한 이유

```
// 위험한 방식 (절대 사용 금지!)
db.exec(`SELECT * FROM todos WHERE id = ${userInput}`);
// 만약 userInput이 "1; DROP TABLE todos" 이면? → 테이블 삭제됨!

// 안전한 방식 (Prepared Statement)
db.prepare('SELECT * FROM todos WHERE id = ?').get(userInput);
// ?에 들어가는 값은 항상 "데이터"로 처리됨 → SQL 인젝션 불가능
```

> **실생활 비유**: 은행 창구에서 **양식**에 맞게 입력하는 것과 같습니다. 양식(prepare)이 정해져 있으면, 아무리 이상한 값을 넣어도 양식을 벗어나지 못합니다.

PostgreSQL 연동

SQLite보다 더 강력한 데이터베이스가 필요할 때 PostgreSQL을 사용합니다.

```
const { Pool } = require('pg');

// 연결 풀 생성 (여러 연결을 미리 만들어두고 재사용)
const pool = new Pool({
  connectionString: process.env.DATABASE_URL
  // 예: "postgresql://user:password@localhost:5432/todo_db"
});

// 조회 (필터링 + 페이지네이션)
router.get('/', async (req, res) => {
  const { done, page = 1, limit = 10 } = req.query;
  const offset = (page - 1) * limit;

  let query = 'SELECT * FROM todos';
  const params = [];

  if (done !== undefined) {
    query += ' WHERE done = $1';  // PostgreSQL은 $1, $2, ... 사용
    params.push(done === 'true');
  }

  query += ` ORDER BY created_at DESC LIMIT $${params.length + 1} OFFSET $${param
  params.push(limit, offset);

  const { rows } = await pool.query(query, params);
  res.json(rows);
});
```

사용자 CRUD + bcrypt

비밀번호는 **절대 평문으로 저장하면 안 됩니다!**

```
const bcrypt = require('bcrypt');

// 회원가입
router.post('/register', async (req, res) => {
  const { email, password } = req.body;

  // 비밀번호 해싱 (원본을 알 수 없는 형태로 변환)
  const hashedPassword = await bcrypt.hash(password, 10);
  // "password123" → "$2b$10$N9qo8uLOickgx2ZMRZoMye..."
  // 10 = salt rounds (높을수록 안전하지만 느림)

  db.prepare('INSERT INTO users (email, password) VALUES (?, ?)').run(email, hash
  res.status(201).json({ message: '회원가입 성공' });
});

// 로그인
router.post('/login', async (req, res) => {
  const { email, password } = req.body;
  const user = db.prepare('SELECT * FROM users WHERE email = ?').get(email);

  if (!user) return res.status(401).json({ error: '이메일 또는 비밀번호가 틀렸습니다' });

  // 입력한 비밀번호와 저장된 해시 비교
  const isMatch = await bcrypt.compare(password, user.password);
  if (!isMatch) return res.status(401).json({ error: '이메일 또는 비밀번호가 틀렸습니다'

  res.json({ message: '로그인 성공', userId: user.id });
});
```

정리

개념	핵심
better-sqlite3	SQLite 동기 방식 (`prepare`, `run`, `get`, `all`)
Prepared Statement	`?` 또는 `$1` 로 SQL 인젝션 방지
COALESCE	NULL이면 기존값 유지 (부분 수정)
pg Pool	PostgreSQL 연결 풀 (비동기)
bcrypt	비밀번호 해싱 (`hash`, `compare`)

Day 29 - ORM: Prisma와 Sequelize

> **Phase 6: Express** | 학습일: 29일차

ORM이란?

ORM(Object-Relational Mapping)은 **SQL을 직접 쓰지 않고** JavaScript 코드로 데이터베이스를 다룰 수 있게 해주는 도구입니다.

> **실생활 비유**: 외국 식당에서 주문할 때
>
> - SQL 직접 작성 = **외국어로 직접 주문** (문법 틀릴 수 있음)
> - ORM 사용 = **통역사가 대신 주문** (나는 한국어로 말하면 됨)

```
SQL 방식:  SELECT * FROM todos WHERE done = 1 ORDER BY created_at DESC
ORM 방식:  prisma.todo.findMany({ where: { done: true }, orderBy: { createdAt: 'desc'
```

Prisma 설정

1. 스키마 정의 (schema.prisma)

```
// prisma/schema.prisma — 데이터베이스 설계도

datasource db {
  provider = "sqlite"              // 어떤 DB를 쓸지
  url      = "file:./dev.db"       // DB 파일 경로
}

generator client {
  provider = "prisma-client-js"    // Prisma Client 생성
}

// 모델 정의 = 테이블 설계
model Todo {
  id        Int      @id @default(autoincrement())  // 자동 증가 ID
  title     String                                   // 필수 문자열
  done      Boolean  @default(false)                 // 기본값 false
  priority  String   @default("medium")              // 기본값 "medium"
  category  String?                                  // ?는 선택적 (null 허용)
  createdAt DateTime @default(now())                 // 생성 시간 자동 기록
  updatedAt DateTime @updatedAt                      // 수정 시 자동 업데이트
}
```

2. 마이그레이션 (테이블 생성)

```
# 스키마를 기반으로 실제 DB 테이블 생성
npx prisma migrate dev --name init

# Prisma Studio (브라우저에서 DB 확인/편집)
npx prisma studio
```

Prisma CRUD

```
const { PrismaClient } = require('@prisma/client');
const prisma = new PrismaClient();

// 전체 조회 (필터 + 정렬 + 페이지네이션)
const todos = await prisma.todo.findMany({
  where: { done: false },            // 조건: 미완료만
  orderBy: { createdAt: 'desc' },    // 정렬: 최신순
  skip: 0,                           // 건너뛰기 (offset)
  take: 10,                          // 가져올 개수 (limit)
});

// 단일 조회
const todo = await prisma.todo.findUnique({
  where: { id: 1 }
});

// 생성
const newTodo = await prisma.todo.create({
  data: {
    title: '프리즈마 배우기',
    priority: 'high',
    category: '학습'
  }
});

// 수정
const updated = await prisma.todo.update({
  where: { id: 1 },
  data: { done: true }
});

// 삭제
await prisma.todo.delete({
  where: { id: 1 }
});
// ⚠️ 없는 id를 삭제하면 에러 발생 (에러 코드: P2025)
```

관계 포함 조회 (include)

```
// 할일과 함께 작성자 정보도 가져오기
const todosWithUser = await prisma.todo.findMany({
  include: { user: true }  // 연결된 User 데이터도 포함
});
// 결과: [{ id: 1, title: '...', user: { id: 1, name: '김철수' } }]
```

SQL vs Prisma 비교

작업	SQL	Prisma
전체 조회	`SELECT * FROM todos`	`prisma.todo.findMany()`
조건 조회	`WHERE done = 1`	`{ where: { done: true } }`
생성	`INSERT INTO todos (title) VALUES (?)`	`prisma.todo.create({ data: { title } })`
수정	`UPDATE todos SET done = 1 WHERE id = ?`	`prisma.todo.update({ where: { id }, data: { done: true } })`
삭제	`DELETE FROM todos WHERE id = ?`	`prisma.todo.delete({ where: { id } })`

시드 데이터

개발/테스트를 위한 초기 데이터를 넣는 것을 "시딩"이라고 합니다.

```
// prisma/seed.js
async function seed() {
  // 대량 생성
  await prisma.todo.createMany({
    data: [
      { title: 'HTML 복습', priority: 'low', category: '학습' },
      { title: 'CSS 실습', priority: 'medium', category: '학습' },
      { title: '장보기', priority: 'high', category: '생활' },
    ]
  });

  // upsert: 있으면 업데이트, 없으면 생성
  await prisma.user.upsert({
    where: { email: 'admin@test.com' },
    update: {},  // 이미 있으면 변경 없음
    create: { email: 'admin@test.com', username: 'admin', password: '...' }
  });
}
```

Sequelize 비교

```
// Sequelize — 또 다른 인기 ORM
const { Sequelize, DataTypes } = require('sequelize');
const sequelize = new Sequelize({
  dialect: 'sqlite',
  storage: './todos.db'
});

// 모델 정의 (Prisma의 schema.prisma 대신 JS 코드로)
const Todo = sequelize.define('Todo', {
  title: { type: DataTypes.STRING, allowNull: false },
  done: { type: DataTypes.BOOLEAN, defaultValue: false },
});

await sequelize.sync();  // 테이블 생성
const todos = await Todo.findAll();  // 전체 조회
```

정리

개념	핵심
ORM	SQL 대신 JS 코드로 DB 다루기
Prisma	스키마 기반, 타입 안전, 마이그레이션
findMany	여러 행 조회 (where, orderBy, skip, take)
create / update / delete	CRUD 메서드
include	관계 데이터 포함 조회
seed	초기 데이터 생성 (createMany, upsert)

Day 30 - 인증: JWT와 bcrypt

> **Phase 6: Express** | 학습일: 30일차

왜 인증이 필요한가?

지금까지 만든 할일 API는 **누구나** 모든 데이터에 접근할 수 있습니다. 실제 서비스에서는 "이 사람이 누구인지" 확인하고, "이 사람의 데이터만" 보여줘야 합니다.

> **실생활 비유**: **놀이공원 입장 팔찌**를 생각해보세요.
>
> 1. 매표소에서 티켓 구매 (회원가입/로그인)
> 2. 입장 팔찌 받기 (JWT 토큰 발급)
> 3. 놀이기구 탈 때마다 팔찌 확인 (인증 미들웨어)
> 4. 자유이용권 vs 일반 (RBAC: 역할에 따라 다른 접근 권한)

JWT (JSON Web Token)

JWT는 서버가 클라이언트에게 발급하는 **디지털 신분증**입니다.

```
토큰 구조: header.payload.signature
            (머리).(내용).(서명)

Header:    { "alg": "HS256", "typ": "JWT" }
Payload:   { "userId": 1, "email": "user@test.com", "iat": 1234567890 }
Signature: HMACSHA256(header + payload, 비밀키)
```

```
const jwt = require('jsonwebtoken');

// 토큰 발급 (로그인 성공 시)
const token = jwt.sign(
  { userId: user.id, email: user.email },  // payload (담을 정보)
  process.env.JWT_SECRET,                   // 비밀키 (서버만 알고 있음)
  { expiresIn: '24h' }                     // 만료 시간
);

// 토큰 검증 (API 요청 시)
try {
  const decoded = jwt.verify(token, process.env.JWT_SECRET);
  console.log(decoded.userId);  // 1
} catch (error) {
  console.log('유효하지 않은 토큰');
}
```

bcrypt — 비밀번호 해싱

비밀번호를 **절대 평문으로 저장하면 안 됩니다!**

> **실생활 비유**: bcrypt 해싱은 **일방통행 도로**와 같습니다.
>
> - "password123" → "$2b$10$N9qo8u..." (해싱: 가능)
> - "$2b$10$N9qo8u..." → "password123" (복원: 불가능!)
>
> 해커가 DB를 탈취해도 원래 비밀번호를 알 수 없습니다.

```
const bcrypt = require('bcrypt');

// 해싱 (회원가입 시)
const hashedPassword = await bcrypt.hash('password123', 10);
// 10 = salt rounds (높을수록 안전하지만 느림, 10이 적당)

// 검증 (로그인 시)
const isMatch = await bcrypt.compare('password123', hashedPassword);
// true → 비밀번호 일치!
```

인증 미들웨어

```javascript
// middleware/auth.js
function authMiddleware(req, res, next) {
  // 1. 헤더에서 토큰 추출
  //    Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
  const authHeader = req.headers.authorization;
  const token = authHeader?.split(' ')[1];  // "Bearer " 다음 부분

  if (!token) {
    return res.status(401).json({ error: '토큰이 없습니다' });
  }

  try {
    // 2. 토큰 검증
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded;  // 요청 객체에 사용자 정보 저장
    next();              // 다음 미들웨어로 이동
  } catch (error) {
    res.status(401).json({ error: '유효하지 않은 토큰입니다' });
  }
}
```

인증 흐름 전체 그림

```
1. 회원가입
   POST /api/auth/register
   → 비밀번호를 bcrypt로 해싱
   → DB에 저장
   → JWT 토큰 발급하여 반환

2. 로그인
   POST /api/auth/login
   → DB에서 사용자 찾기
   → bcrypt.compare로 비밀번호 확인
   → JWT 토큰 발급하여 반환

3. API 사용 (인증 필요)
   GET /api/todos
   → Authorization: Bearer <토큰>
   → authMiddleware가 토큰 검증
   → req.user.userId로 본인 데이터만 조회
```

회원가입/로그인 라우트

```
// routes/auth.js
router.post('/register', async (req, res) => {
  const { username, email, password } = req.body;

  // 1. 이메일 중복 확인
  const existing = await prisma.user.findUnique({ where: { email } });
  if (existing) return res.status(409).json({ error: '이미 사용 중인 이메일입니다' });

  // 2. 비밀번호 해싱
  const hashedPassword = await bcrypt.hash(password, 10);

  // 3. 사용자 생성
  const user = await prisma.user.create({
    data: { username, email, password: hashedPassword }
  });

  // 4. 토큰 발급
  const token = jwt.sign({ userId: user.id }, process.env.JWT_SECRET, { expiresIn
  res.status(201).json({ token, user: { id: user.id, username, email } });
});

router.post('/login', async (req, res) => {
  const { email, password } = req.body;

  // 1. 사용자 찾기
  const user = await prisma.user.findUnique({ where: { email } });
  if (!user) return res.status(401).json({ error: '이메일 또는 비밀번호가 틀렸습니다' });

  // 2. 비밀번호 확인
  const isMatch = await bcrypt.compare(password, user.password);
  if (!isMatch) return res.status(401).json({ error: '이메일 또는 비밀번호가 틀렸습니다'

  // 3. 토큰 발급
  const token = jwt.sign({ userId: user.id }, process.env.JWT_SECRET, { expiresIn
  res.json({ token, user: { id: user.id, username: user.username, email } });
});
```

RBAC (역할 기반 접근 제어)

> **실생활 비유**: 회사의 **직급별 출입 카드**와 같습니다.
>
> - 사원: 사무실만 출입 가능
> - 팀장: 사무실 + 회의실
> - 임원: 모든 곳 출입 가능

```
// middleware/rbac.js
function requireRole(...roles) {
  return (req, res, next) => {
    if (!roles.includes(req.user.role)) {
      return res.status(403).json({ error: '접근 권한이 없습니다' });
    }
    next();
  };
}

// 사용 예시
// admin만 사용자 삭제 가능
router.delete('/users/:id', authMiddleware, requireRole('admin'), deleteUser);

// admin 또는 manager만 통계 조회 가능
router.get('/stats', authMiddleware, requireRole('admin', 'manager'), getStats);
```

Rate Limiting (API 호출 제한)

악의적인 대량 요청(DDoS)을 방지합니다.

```
const rateLimit = require('express-rate-limit');

const limiter = rateLimit({
  windowMs: 15 * 60 * 1000,  // 15분
  max: 100,                  // 최대 100회
  message: { error: '요청이 너무 많습니다. 잠시 후 다시 시도해주세요.' }
});

app.use('/api/', limiter);
```

정리

개념	핵심
JWT	토큰 기반 인증 (`jwt.sign` , `jwt.verify`)
bcrypt	비밀번호 해싱 (일방향, 복원 불가)
인증 미들웨어	`Authorization: Bearer <token>` 검증
RBAC	역할별 접근 제어 (`requireRole('admin')`)
Rate Limiting	API 호출 횟수 제한

Day 31 - API 문서: Swagger

> **Phase 6: Express** | 학습일: 31일차

왜 API 문서가 필요한가?

API를 만들었지만, 다른 개발자(특히 프론트엔드 개발자)가 사용하려면 **사용 설명서**가 필요합니다.

> **실생활 비유**: 가전제품을 샀는데 **사용 설명서**가 없다면?
>
> - "이 버튼은 뭐지?" → API 엔드포인트가 뭐하는 건지 모름
> - "건전지는 어디에 넣지?" → 어떤 데이터를 보내야 하는지 모름
> - Swagger = **인터랙티브 사용 설명서** (직접 눌러볼 수 있음!)

Swagger 설정

```
// src/swagger.js
const swaggerJsdoc = require('swagger-jsdoc');

const swaggerSpec = swaggerJsdoc({
  definition: {
    openapi: '3.0.0',
    info: {
      title: '할일 관리 API',
      version: '1.0.0',
      description: 'Express + Prisma로 만든 할일 관리 REST API',
    },
    components: {
      securitySchemes: {
        bearerAuth: {
          type: 'http',
          scheme: 'bearer',
          bearerFormat: 'JWT',
        },
      },
    },
  },
  apis: ['./src/routes/*.js'],  // JSDoc 주석을 읽을 파일 경로
});

module.exports = swaggerSpec;
```

```
// src/index.js — Swagger UI 연결
const swaggerUi = require('swagger-ui-express');
const swaggerSpec = require('./swagger');

app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(swaggerSpec));
// → http://localhost:3000/api-docs 에서 문서 확인!
```

JSDoc으로 API 문서 작성

라우트 코드 위에 주석을 달면 자동으로 문서가 생성됩니다.

```
/**
 * @swagger
 * /api/todos:
 *   get:
 *     summary: 할일 목록 조회
 *     tags: [Todos]
 *     security:
 *       - bearerAuth: []
 *     parameters:
 *       - in: query
 *         name: done
 *         schema:
 *           type: boolean
 *         description: 완료 여부로 필터링
 *     responses:
 *       200:
 *         description: 성공
 *         content:
 *           application/json:
 *             schema:
 *               type: array
 *               items:
 *                 $ref: '#/components/schemas/Todo'
 *       401:
 *         description: 인증 필요
 */
router.get('/', authMiddleware, async (req, res) => {
  // ... 실제 구현
});
```

```
/**
 * @swagger
 * /api/todos:
 *   post:
 *     summary: 새 할일 추가
 *     tags: [Todos]
 *     security:
 *       - bearerAuth: []
 *     requestBody:
 *       required: true
 *       content:
 *         application/json:
 *           schema:
 *             type: object
 *             required:
 *               - title
 *             properties:
 *               title:
 *                 type: string
 *                 example: "Express 공부하기"
 *               priority:
 *                 type: string
 *                 enum: [low, medium, high]
 *                 example: "high"
 *     responses:
 *       201:
 *         description: 생성 성공
 *       400:
 *         description: 잘못된 요청
 */
router.post('/', authMiddleware, async (req, res) => {
  // ... 실제 구현
});
```

정리

개념	핵심
OpenAPI	REST API 문서 표준 규격
swagger-jsdoc	JSDoc 주석 → API 명세 자동 생성
swagger-ui-express	브라우저에서 인터랙티브 API 문서
@swagger	라우트 위에 주석으로 명세 작성
bearerAuth	JWT 토큰 인증 스킴
/api-docs	Swagger UI 접속 경로

Day 32 - 풀스택 완성: React + Express

> **Phase 6: Express** | 학습일: 32일차

드디어 풀스택!

지금까지 배운 모든 것을 합칩니다:

- **프론트엔드**: React + TypeScript + Context API + React Router
- **백엔드**: Express + Prisma + JWT 인증 + Swagger
- **연동**: CORS + Vite 프록시 + fetch API

> **실생활 비유**: 지금까지 우리는 **주방(백엔드)**과 **홀(프론트엔드)**을 따로 만들었습니다. 이제 둘을 연결해서 **실제 운영 가능한 식당**을 완성합니다!

CORS (교차 출처 리소스 공유)

React와 Express는 다른 포트에서 실행됩니다. 브라우저는 보안상 다른 출처(origin)의 API를 차단합니다.

```
React:   http://localhost:5173  ← 프론트엔드
Express: http://localhost:3000  ← 백엔드

→ 포트가 다르면 "다른 출처" = 브라우저가 차단!
```

> **실생활 비유**: CORS는 **다른 나라에서 온 택배의 통관 절차**와 같습니다.
>
> - 같은 나라(같은 출처) → 통관 없이 바로 배달
> - 다른 나라(다른 출처) → 통관 허가(CORS 설정) 필요

해결 방법 1: Express에서 CORS 허용

```
const cors = require('cors');

app.use(cors({
  origin: 'http://localhost:5173',  // React 개발 서버 주소
  credentials: true                  // 쿠키 포함 허용
}));
```

해결 방법 2: Vite 프록시 (개발 환경 추천)

```
// vite.config.ts
export default defineConfig({
  server: {
    proxy: {
      '/api': 'http://localhost:3000'
      // React에서 /api/todos 요청 → Vite가 대신 http://localhost:3000/api/todos로 전달
      // 브라우저 입장에서는 같은 출처이므로 CORS 문제 없음!
    }
  }
});
```

API 호출 함수 작성

```typescript
// client/src/api/auth.ts
export async function login(email: string, password: string) {
  const res = await fetch('/api/auth/login', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ email, password }),
  });

  if (!res.ok) {
    const error = await res.json();
    throw new Error(error.error || '로그인 실패');
  }

  return res.json();  // { token, user }
}

// client/src/api/todos.ts
export async function fetchTodos(token: string) {
  const res = await fetch('/api/todos', {
    headers: {
      Authorization: `Bearer ${token}`,  // 인증 토큰 포함
    },
  });
  return res.json();
}

export async function createTodo(title: string, token: string) {
  const res = await fetch('/api/todos', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      Authorization: `Bearer ${token}`,
    },
    body: JSON.stringify({ title }),
  });
  return res.json();
}
```

AuthContext (토큰 전역 관리)

```tsx
// client/src/context/AuthContext.tsx
'use client';
import { createContext, useContext, useState, useEffect, ReactNode } from 'react'

interface AuthContextType {
  token: string | null;
  user: { id: number; username: string; email: string } | null;
  login: (token: string, user: any) => void;
  logout: () => void;
}

const AuthContext = createContext<AuthContextType | null>(null);

export function AuthProvider({ children }: { children: ReactNode }) {
  // localStorage에서 토큰 복원 (새로고침해도 로그인 유지)
  const [token, setToken] = useState<string | null>(
    () => localStorage.getItem('token')
  );
  const [user, setUser] = useState<any>(
    () => JSON.parse(localStorage.getItem('user') || 'null')
  );

  const login = (newToken: string, newUser: any) => {
    setToken(newToken);
    setUser(newUser);
    localStorage.setItem('token', newToken);
    localStorage.setItem('user', JSON.stringify(newUser));
  };

  const logout = () => {
    setToken(null);
    setUser(null);
    localStorage.removeItem('token');
    localStorage.removeItem('user');
  };

  return (
    <AuthContext.Provider value={{ token, user, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}

export function useAuth() {
  const context = useContext(AuthContext);
  if (!context) throw new Error('useAuth는 AuthProvider 안에서 사용해야 합니다');
  return context;
}
```

풀스택 아키텍처 전체 그림

```
client/ (React + Vite)                server/ (Express + Prisma)
├── src/                               ├── src/
│   ├── api/                           │   ├── routes/
│   │   ├── auth.ts  ── fetch ──→      │   │   ├── auth.js
│   │   └── todos.ts ── fetch ──→      │   │   └── todos.js
│   ├── context/                       │   ├── middleware/
│   │   └── AuthContext.tsx            │   │   ├── auth.js
│   ├── pages/                         │   │   └── rbac.js
│   │   ├── LoginPage.tsx              │   └── swagger.js
│   │   ├── RegisterPage.tsx           ├── prisma/
│   │   └── TodosPage.tsx              │   └── schema.prisma
│   └── components/                    └── .env
│       ├── Navbar.tsx
│       └── TodoItem.tsx
└── vite.config.ts (프록시 설정)
```

실행 방법

```
# 터미널 1: 서버 실행
cd server
npm install
npm run db:migrate
npm run dev
# → http://localhost:3000 (API)
# → http://localhost:3000/api-docs (Swagger 문서)

# 터미널 2: 클라이언트 실행
cd client
npm install
npm run dev
# → http://localhost:5173 (React 앱)
```

32일간의 여정 정리

```
Phase 1: JavaScript 기초 (Day 01~05)
  변수, 함수, 객체, 비동기, DOM

Phase 2: TypeScript (Day 06~08)
  타입 안전성, 인터페이스, 제네릭

Phase 3: React (Day 09~15)
  컴포넌트, 상태 관리, 라우팅, Context

Phase 4: Next.js (Day 16~20)
  SSR, App Router, API Routes

Phase 5: Database (Day 21~23)
  SQL, 모델링, JOIN

Phase 6: Express (Day 24~32)
  Node.js, Express, DB 연동, ORM, 인증, API 문서, 풀스택
```

정리

개념	핵심
CORS	다른 출처 간 리소스 공유 허용
Vite 프록시	개발 환경 CORS 우회 (`/api` → 백엔드)
fetch + token	`Authorization: Bearer <token>`
AuthContext	토큰/사용자 정보 전역 관리
풀스택	React + Express + Prisma + JWT

> **축하합니다!** 32일간의 웹 풀스택 과정을 완료했습니다. 이제 여러분은 프론트엔드부터 백엔드까지, 데이터베이스부터 인증까지 — 웹 애플리케이션의 모든 부분을 직접 만들 수 있습니다!